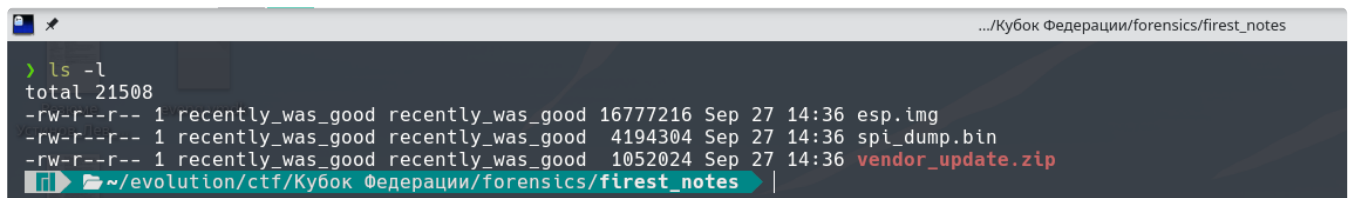


Описание

Ноутбук из бухгалтерии уехал в утилизацию, но перед этим с него сняли полный дампы SPI-флеша и образ EFI-раздела. Пользователь клянётся, что ставил только "официальное" обновление BIOS с портала вендора. Инцидент-репорт тем временем упоминает, что после апдейта на машине один раз заглянули в `ledger.cfg`, которого там давно нет. Проверьте сами.

Файлы задания



```
.../Кубок Федерации/forensics/firest_notes  
$ ls -l  
total 21508  
-rw-r--r-- 1 recently_was_good recently_was_good 16777216 Sep 27 14:36 esp.img  
-rw-r--r-- 1 recently_was_good recently_was_good 4194304 Sep 27 14:36 spi_dump.bin  
-rw-r--r-- 1 recently_was_good recently_was_good 1052024 Sep 27 14:36 vendor_update.zip
```

Решение

1. Смотрим EFI-раздел

Итак, начнём. В условии прямо назван файл `ledger.cfg`, поэтому первым делом идём в образ EFI-раздела. Чтобы не возиться с монтированием, посмотрим его содержимое утилитой `7z` — она умеет читать FAT-образы напрямую:

```
7z l esp.img
```

Разберём команду:

- `l` (list) — это не флаг, а команда `7z`: просто вывести список файлов, ничего не распаковывая.

```
.../Кубок Федерации/forensics/firest_notes

> 7z l esp.img

7-Zip 26.03 (x64) : Copyright (c) 1999-2026 Igor Pavlov : 2026-09-03
64-bit locale=en_US.UTF-8 Threads:8 OPEN_MAX:4096, ASM

Scanning the drive for archives:
1 file, 16777216 bytes (16 MiB)

Listing archive: esp.img
--
Path = esp.img
Type = FAT
Physical Size = 16777216
File System = FAT16
Cluster Size = 2048
Free Space = 15843328
Headers Size = 55296
Modified = 2026-08-25 23:12:56
Label = FUNDFUND
Sector Size = 512
ID = 1955845736

  Date       Time       Attr      Size   Compressed  Name
  -----
2026-08-25 23:12:56 D....           233        2048  EFI
2026-08-25 23:26:12 ....A           233        2048  ledger.cfg
2026-08-25 23:12:56 D....           233        2048  EFI/BOOT
2026-08-25 23:18:50 ....A      875456      876544  EFI/BOOT/BOOTX64.EFI
  -----
2026-08-25 23:26:12           875689      878592  2 files, 2 folders

~/evolution/ctf/Кубок Федерации/forensics/firest_notes
```

Кроме стандартного загрузчика в корне раздела лежит тот самый `ledger.cfg`. Достанем его:

```
7z e esp.img ledger.cfg -o./out
cat esp_out/ledger.cfg
```

Разберём команду:

- `e` (extract) — распаковать файл без сохранения структуры каталогов (с каталогами было бы `x`);
- `ledger.cfg` — что именно достаём, чтобы не тащить весь образ;
- `-o./out` — каталог, куда положить результат (пишется слитно; если каталога нет, `7z` создаст его сам).

```
> cat ledger.cfg
# ledger record - fund 1147, opis 1, delo 9
[esp]
bootorder=hd,internal-shell
console=serial,115200
recovery_pin=1147-0091
note=digitization batch 9: originals sealed in the depository
first_record=REDACTED-TRANSFERRED-TO-MAIN-VAULT

~/evolution/ctf/Кубок Федерации/forensics/firest_notes/out
```

И вот тут уже интересно, разберём построчно:

- `# ledger record - fund 1147, opis 1, delo 9` — архивный шифр: «фонд 1147, описание 1, дело 9». Число 1147 запоминаем, оно ещё встретится.

- `[esp]` — просто имя секции. ESP = EFI System Partition, тот самый раздел, образ которого мы смотрим.
- `bootorder=hd,internal-shell` — порядок загрузки: сначала диск, потом встроенный UEFI Shell.
- `console=serial,115200` — консоль выведена на последовательный порт, скорость 115200 бод.
- `recovery_pin=1147-0091` — «PIN восстановления», собранный из тех же архивных чисел. Выглядит как приманка, и дальше это подтвердится.
- `note=digitization batch 9: originals sealed in the depository` — «партия оцифровки 9: оригиналы опечатаны в хранилище».
- `first_record=REDACTED-TRANSFERRED-TO-MAIN-VAULT` — самое важное: значение вырезано, «первая запись перенесена в главное хранилище».

То есть на разделе лежит уже отредактированная версия файла, а оригинальное значение `first_record` куда-то «перенесли». На ESP его больше нет, значит, *главное хранилище* надо искать в прошивке.

2. Обновление от вендора и карта дампа

Посмотрим, что лежит в «официальном» обновлении, и распакуем его с помощью:

```
7z l vendor_update.zip
7z x vendor_update.zip -ouupdate
ls -l spi_dump.bin update/
```

Разберём команду:

- `x` — распаковать с сохранением путей (в отличие от `e`);
- `-ouupdate` — складываем всё в каталог `update`.

```

> 7z l vendor_update.zip

7-Zip 26.03 (x64) : Copyright (c) 1999-2026 Igor Pavlov : 2026-09-03
64-bit locale=en_US.UTF-8 Threads:8 OPEN_MAX:4096, ASM

Scanning the drive for archives:
1 file, 1052024 bytes (1028 KiB)

Listing archive: vendor_update.zip
--
Path = vendor_update.zip
Type = zip
Physical Size = 1052024
Разрешение
CTF Date Time Attr Size Compressed Name
-----
2026-08-26 01:30:40 ..... 3653632 1050689 OVMF_CODE.fd
2026-08-26 01:30:40 ..... 540672 780 OVMF_VARS.fd
2026-08-26 01:30:40 ..... 125 115 RELEASE_NOTES.txt
-----
2026-08-26 01:30:40 4194429 1051584 3 files
> 7z x vendor_update.zip -ouupdate

7-Zip 26.03 (x64) : Copyright (c) 1999-2026 Igor Pavlov : 2026-09-03
64-bit locale=en_US.UTF-8 Threads:8 OPEN_MAX:4096, ASM

Scanning the drive for archives:
1 file, 1052024 bytes (1028 KiB)

Extracting archive: vendor_update.zip
--
Path = vendor_update.zip
Type = zip
Physical Size = 1052024

Everything is Ok

Files: 3
Size: 4194429
Compressed: 1052024
> ls -l spi_dump.bin update/
-rw-r--r-- 1 recently_was_good recently_was_good 4194304 Sep 27 14:36 spi_dump.bin

update/:
total 4100
-rw-r--r-- 1 recently_was_good recently_was_good 3653632 Aug 26 01:30 OVMF_CODE.fd
-rw-r--r-- 1 recently_was_good recently_was_good 540672 Aug 26 01:30 OVMF_VARS.fd
-rw-r--r-- 1 recently_was_good recently_was_good 125 Aug 26 01:30 RELEASE_NOTES.txt

```

Внутри `OVMF_CODE.fd` (3653632 байт), `OVMF_VARS.fd` (540672 байт) и `RELEASE_NOTES.txt`. OVMF — это UEFI-прошивка для виртуальных машин QEMU, так что наш «ноутбук» на самом деле виртуалка. А теперь сложим размеры:

```

echo $((3653632 + 540672)) # 4194304
printf '%#x\n' 3653632 # 0x37c000

```

```

> echo $((3653632 + 540672))
4194304
> printf '%#x\n' 3653632
0x37c000

```

Получается ровно размер `spi_dump.bin` — 4194304 байт (4 МБ). Никуда ничего не «пропало»: дампы флеша это просто два файла, склеенные подряд. Отсюда карта дампа:

Область	Диапазон	Размер	Что там
CODE	0x000000 — 0x37C000	3653632	исполняемый код прошивки (как <code>OVMF_CODE.fd</code>)
VARS	0x37C000 — 0x400000	540672	NVRAM — хранилище UEFI-переменных (как <code>OVMF_VARS.fd</code>)

И заодно у нас появился эталон: файлы из архива — это «чистая» прошивка, с которой можно сравнивать дамп.

3. Сравниваем прошивку с эталоном

Сравнивать побайтово бессмысленно: почти все драйверы лежат внутри тома, сжатого LZMA, и один лишний модуль меняет весь сжатый поток. Поэтому сравнивать будем на уровне модулей. Для этого возьмём `uefiextract` (консольная утилита из набора UEFITool) она сама распакует вложенные тома и выпишет все модули с их GUID и именами:

```
uefiextract spi_dump.bin guids
uefiextract update/OVMF_CODE.fd guids
diff <(sort update/OVMF_CODE.fd.guids.csv) <(sort spi_dump.bin.guids.csv)
```

Разберём команды:

- `guids` — режим `uefiextract`: ничего не распаковывать на диск, только сохранить список GUID, имя в файл `<образ>.guids.csv`;
- `sort` — сортируем оба списка, чтобы порядок модулей не мешал сравнению;
- `diff <(...) <(...)` — сравниваем вывод двух команд, как будто это два файла; строки с `>` есть только в дампе.

```
> uefiextract spi_dump.bin guids
> uefiextract update/OVMF_CODE.fd guids
> diff <(sort update/OVMF_CODE.fd.guids.csv) <(sort spi_dump.bin.guids.csv)
29a30
> 3A91C7E2-5D64-4F0B-8A19-C7E2D5B40F36,BoardSkuDxe
48a50
> 6F3D9A41-2C8B-4E17-B05A-93D7C4E21F88,LedgerDxe
79a82
> B14E8D5F-6A23-4C97-9F02-1D5E8B3A6C44,ChassisTagDxe
~/evolution/ctf/Кубок Федерации/forensics/firest_notes
```

Вот что должно насторожить: в реальной прошивке сидят три DXE-драйвера, которых в «официальном» обновлении нет вообще. То есть пользователь, может, и ставил обновление с портала, но во флеше лежит не оно. И название `LedgerDxe` слишком уж хорошо совпадает с `ledger.cfg`.

4. NVRAM: ищем «главное хранилище»

Раз мы знаем, что с `0x37C000` начинается NVRAM, заглянем туда:

```
xxd -s 3653632 -l 128 spi_dump.bin
```

Разберём команду:

- `xxd` — утилита для просмотра hex-дампа;
- `-s 3653632` — смещение, с которого начать (можно и в hex: `-s 0x37c000`);
- `-l 128` — сколько байт прочитать.

```
> xxd -s 3653632 -l 128 spi_dump.bin
0037c000: 0000 0000 0000 0000 0000 0000 0000 0000 .....
0037c010: 8d2b f1ff 9676 8b4c a985 2747 075b 4f50 .+...v.L...'G.[0P
0037c020: 0040 0800 0000 0000 5f46 5648 fffe 0400 .@....._FVH...
0037c030: 4800 afb8 0000 0002 8400 0000 0010 0000 H.....
0037c040: 0000 0000 0000 0000 782c f3aa 7b94 9a43 .....x,..{..C
0037c050: a180 2e14 4ec3 7792 b8ff 0300 5afe 0000 ....N.w....Z...
0037c060: 0000 0000 aa55 3c00 0700 0000 0000 0000 ....U<.....
0037c070: 0000 0000 0000 0000 0000 0000 0000 0000 .....
~/evolution/ctf/Кубок Федерации/forensics/firest_notes
```

Теория подтвердилась. По смещению `+0x28` видим сигнатуру `_FVH` — это заголовок Firmware Volume, а GUID перед ней (`8D 2B F1 FF ... = FFF12B8D-7696-4C8B-A985-2747075B4F50`) говорит, что это именно том с NV-данными. Сразу за заголовком тома идёт заголовок хранилища переменных, а дальше — сами переменные одна за другой, каждая начинается с маркера `AA 55`.

Имена UEFI-переменных хранятся в UTF-16, поэтому обычный `strings` их не увидит. Поищем всё, что связано с ledger:

```
strings -e l -t x spi_dump.bin | grep -i ledger
```

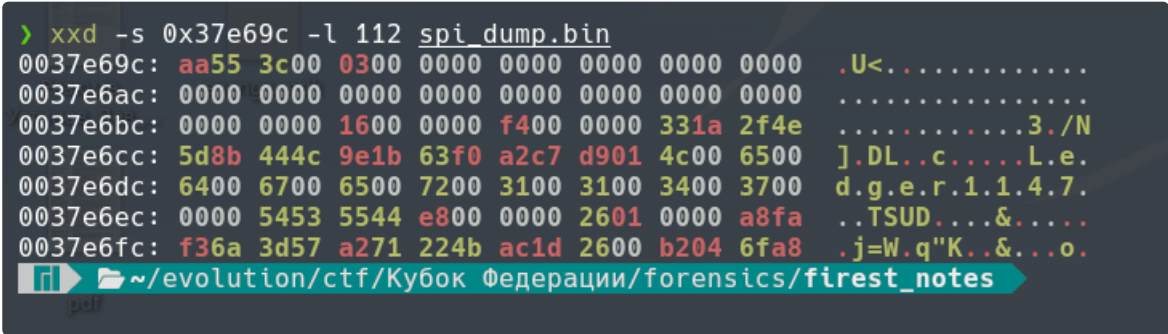
Разберём команду:

- `-e l` — искать строки в 16-битной кодировке little-endian (UTF-16LE);
- `-t x` — печатать перед каждой строкой её смещение в hex;
- `grep -i` — фильтр без учёта регистра.

```
> strings -e l -t x spi_dump.bin | grep -i ledger
37e6d8 Ledger1147
37f614 Ledger1147
~/evolution/ctf/Кубок Федерации/forensics/firest_notes
```

И тут раскрываются глаза: в NVRAM лежит переменная `Ledger1147` (то самое число из конфига), причём в двух экземплярах. Так бывает, потому что хранилище переменных ничего не перезаписывает на месте: при изменении новая версия дописывается в конец, а старая просто помечается удалённой и лежит во флеше до следующей сборки мусора. Чтобы понять, какая из копий активна, а какая удалена, посмотрим пошире — вместе с заголовком переменной:

```
xxd -s 0x37e69c -l 112 spi_dump.bin
```



Смещение	Байты	Поле	Значение
0x37E69C	AA 55	StartId	маркер начала переменной
0x37E69E	3C	State	удалена
0x37E6A0	03 00 00 00	Attributes	NV + BootService
0x37E6C0	16 00 00 00	NameSize	22 байта (10 символов UTF-16 + ноль)
0x37E6C4	F4 00 00 00	DataSize	244 байта
0x37E6C8	33 1A 2F 4E ...	VendorGuid	4E2F1A33-8B5D-4C44-9E1B-63F0A2C7D901
0x37E6D8	4C 00 65 00 ...	Name	Ledger1147
0x37E6EE	54 53 55 44 ...	Data	TSUD и дальше «шум»

Поле State: `0x3F` переменная действующая, `0x3C` удалённая. Чтобы не разбирать вторую копию руками, напомним маленький скрипт, который найдёт все копии по VendorGuid:

```
# nvram_ledger.py - ищем все копии переменной Ledger1147 в NVRAM
import uuid

data = open("spi_dump.bin", "rb").read()
guid = uuid.UUID("4e2f1a33-8b5d-4c44-9e1b-63f0a2c7d901").bytes_le # VendorGuid из
hex-дампа
STATE = {0x3F: "valid", 0x3C: "deleted"}

i = data.find(guid)
while i != -1:
    hdr = data.rfind(b"\xaa\x55", 0, i) # StartId переменной
    (0x55AA)
    state = data[hdr + 2]
    ns = int.from_bytes(data[i - 8 : i - 4], "little") # NameSize
    ds = int.from_bytes(data[i - 4 : i], "little") # DataSize
```

```

name = data[i + 16 : i + 16 + ns].decode("utf-16-le").rstrip("\0")
body = i + 16 + ns
print(f"GUID @ {i:#x} | header @ {hdr:#x} | state={state:#04x}
({STATE.get(state, '?')}) "
      f"| {name} | NameSize={ns} DataSize={ds} | data @ {body:#x} | magic=
{data[body:body+4]}")
i = data.find(guid, i + 1)

```

```

> python first_note.py
GUID @ 0x37e6c8 | header @ 0x37e69c | state=0x3c (deleted) | Ledger1147 | NameSize=22 DataSize=244 | data @ 0x37e6ee | magic=b'TSUD'
GUID @ 0x37f604 | header @ 0x37f5d8 | state=0x3f (valid) | Ledger1147 | NameSize=22 DataSize=245 | data @ 0x37f62a | magic=b'TSUD'
~/evolution/ctf/Кубок Федерации/forensics/firest_notes

```

Итого две ревизии:

Копия	State	DataSize	Полезная нагрузка
0x37E6C8	0x3C — удалена	244	232 байта
0x37F604	0x3F — действует	245	233 байта

Данные обеих копий устроены одинаково: 4 байта TSUD , затем длина (E8 00 00 00 = 232), затем 26 01 00 00 , и дальше сплошной шум. Это не кодировка, а шифртекст.

Теперь самое вкусное. Действующая копия 233 байта, и ровно столько же весит нынешний ledger.cfg (wc -c esp_out/ledger.cfg). А удалённая на байт короче. Строка REDACTED-TRANSFERRED-T0-MAIN-VAULT это 34 символа, значит, в первой версии файла на её месте было значение из 33 символов. Вот она, первая запись: лежит в удалённой переменной. Осталось расшифровать, а для этого нужен тот, кто её туда положил.

5. Достаём LedgerDxe

Вытащим подозрительный драйвер из дампа по его GUID:

```

uefiextract spi_dump.bin 6F3D9A41-2C8B-4E17-B05A-93D7C4E21F88 -m body -t 10
mv spi_dump.bin.dump/body.bin LedgerDxe.efi
file LedgerDxe.efi

```

Разберём команду:

- GUID после имени образа — достать только файл с этим GUID (писать заглавными буквами);
- -m body — сохранить тело без заголовков;
- -t 10 — взять секцию типа 0x10 , то есть PE32-образ, сам исполняемый файл драйвера;
- результат падает в spi_dump.bin.dump/body.bin (каталога spi_dump.bin.dump до запуска быть не должно).

```
> uefiextract spi_dump.bin 6F3D9A41-2C8B-4E17-B05A-93D7C4E21F88 -m body -t 10
> mv spi_dump.bin.dump/body.bin LedgerDxe.efi
> file LedgerDxe.efi
LedgerDxe.efi: PE32+ executable for EFI (boot service driver), x86-64, 2 sections
~/.evolution/ctf/Кубок Федерации/forensics/firest_notes
```

6. Реверс LedgerDxe

Открываем драйвер в дизассемблере Ghidra и нас сразу встречает нужна функция:

```
Decompile: entry - (LedgerDxe.efi)
1
2 /* WARNING: Globals starting with '_' overlap smaller symbols at the same
3
4 void entry(undefined8 param_1, longlong param_2)
5
6 {
7     undefined1 local_10 [8];
8
9     if (DAT_000004e1 == '\0') {
10         DAT_000004e1 = '\x01';
11         DAT_000004e0 = DAT_0080b000;
12     }
13     DAT_00001500 = *(longlong *)(param_2 + 0x60);
14     DAT_000004f0 = *(undefined8 *)(param_2 + 0x58);
15     _DAT_000004e8 = param_2;
16     (*(code **)(DAT_00001500 + 0x170))(0x200, 8, 0x240, 0, 0x4c0, local_10);
17     return;
18 }
19
```

Точка входа совсем короткая: драйвер запоминает таблицы `BootServices` / `RuntimeServices` и регистрирует колбэк на событие `ReadyToBoot`:

```
0000023e 00      ??      00h
0000023f 00      ??      00h
//
// .text
// ram:00000240-ram:000004bf
//
LAB_00000240                                XREF[2]: 000000ac(*), 00000194(*)
00000240 55      PUSH     RBP
00000241 74      JZ       LAB_00000242
```

`ReadyToBoot` это момент прямо перед запуском загрузчика: файловые системы уже доступны, запись в NVRAM ещё работает. То есть вся полезная нагрузка обрабатывается при каждой загрузке машины. Смотрим сам колбэк:

```
void ReadyToBootCallback(void)
{
    // LocateProtocol: ищем файловую систему (GUID по адресу 0x4d0)
    status = (*(code **)(gBS + 0x140))(0x4d0, 0, &sfs);
    if (-1 < status) {
        // OpenVolume: открываем корень ESP
        status = (*(code **)(sfs + 8))(sfs, &root);
        if (-1 < status) {
            // Open: 0x456 = L"ledger.cfg", 1 = EFI_FILE_MODE_READ
        }
    }
}
```

```

status = (**(code **)(root + 8))(root, &file, 0x456, 1, 0);
if (status < 0) {
    (**(code **)(root + 0x10))(root);
}
else {
    // Read: читаем файл в буфер 0x50c, не больше 0xff4 байт
    size = 0xff4;
    status = (**(code **)(file + 0x20))(file, &size, 0x50c);
    (**(code **)(file + 0x10))(file);
    (**(code **)(root + 0x10))(root);
    if (-1 < status) {
        if (size != 0) {
            //seed из 16 байт ключа по адресу 0x490
            i = 0;
            st = 0x9e3779b97f4a7c15;
            do {
                pb = (byte *)(i + 0x490);
                i = i + 1;
                st = st << 8 | (ulonglong)*pb;
            } while (i != 0x10);
            //начальное состояние зависит от длины файла
            //      -0x2e4ab5cd2e6d12fd == 0xd1b54a32d192ed03
            i = 0;
            st = size * -0x2e4ab5cd2e6d12fd ^ st;
            st = st ^ st >> 0x1b;
            //гамма и XOR поверх буфера
            do {
                st = st >> 0xc ^ st;
                st = st ^ st << 0x19;
                st = st >> 0x1b ^ st;
                *(byte *)(i + 0x50c) =
                    *(byte *)(i + 0x50c) ^ (byte)(st * 0x2545f4914f6cdd1d >> 0x38);
                i = i + 1;
            } while (size != i);
            // заголовок: TSUD длина 0x126
            _DAT_00000504 = (undefined4)size;
            _DAT_00000500 = 0x44555354;
            _DAT_00000508 = 0x126;
            // SetVariable(L"Ledger1147" @0x46c, GUID @0x4a0, NV|BS, size+12, 0x500)
            (**(code **)(gRT + 0x58))(0x46c, 0x4a0, 3, size + 0xc, 0x500);
        }
    }
}
}
}
}
return;
}

```

Разберём, что тут происходит:

- драйвер при каждой загрузке читает `ledger.cfg` с EFI-раздела;
- шифрует его самописной гаммой: генератор типа xorshift, байт гаммы — старший байт состояния, умноженного на константу;

- начальное состояние считается из ключа и **длины файла**, поэтому у файлов разной длины гамма разная;
- дописывает заголовок `TSUD` и кладёт всё в UEFI-переменную `Ledger1147`.

```
0x10490: 5A C3 11 7E 94 2B D8 60 1F A9 45 CC 73 E6 08 BB ; key_blob
0x104A0: 33 1A 2F 4E 5D 8B 44 4C 9E 1B 63 F0 A2 C7 D9 01 ; VendorGuid
```

GUID по адресу `0x104A0` побайтово совпадает с тем, что мы видели в заголовке переменной в NVRAM, значит, это точно тот драйвер. Вот и *главное хранилище* копия `ledger.cfg` уезжает во флеш материнской платы и переживает и удаление файла, и переустановку системы. XOR симметричен, так что той же функцией данные и расшифровываются.

Два остальных драйвера обманки. Устроены так же, но их колбэк состоит из одной строки:

`BoardSkuDxe` пишет переменную `BoardSku = "SKU-BKUP01-REV2-BATCH07"`, а `ChassisTagDxe` `ChassisTag = "CHT-0091-A"`. Тот самый `0091` из `recovery_pin`, который вёл в никуда.

7. Расшифровка

Переносим алгоритм в Python и прогоняем через него обе копии переменной:

```
import uuid

M = (1 << 64) - 1
KEY = bytes.fromhex("5ac3117e942bd8601fa945cc73e608bb")

def crypt(buf):
    seed = int.from_bytes(KEY[8:16], "big")
    st = ((0xD1B54A32D192ED03 * len(buf)) & M) ^ seed
    st ^= st >> 27
    out = bytearray()
    for b in buf:
        t = st ^ (st >> 12)
        t = t ^ ((t << 25) & M)
        st = t ^ (t >> 27)
        out.append(b ^ ((st * 0x2545F4914F6CDD1D) & M) >> 56))
    return bytes(out)

data = open("spi_dump.bin", "rb").read()
guid = uuid.UUID("4e2f1a33-8b5d-4c44-9e1b-63f0a2c7d901").bytes_le

i = data.find(guid)
while i != -1:
    ns = int.from_bytes(data[i - 8 : i - 4], "little")
    ds = int.from_bytes(data[i - 4 : i], "little")
    blob = data[i + 16 + ns : i + 16 + ns + ds]
    print(f"===== GUID @ {i:#x}, payload {len(blob) - 12} байт =====")
    print(crypt(blob[12:]).decode(errors="replace"))
    i = data.find(guid, i + 1)
```

Разберём скрипт:

- `& M` — в Python числа бесконечной длины, поэтому после умножений и сдвигов влево обрезаем результат до 64 бит, как в регистре;
- `blob[12:]` — пропускаем 12 байт заголовка (TSUD, длина, служебное поле);
- цикл по `data.find(guid)` — проходим по всем копиям переменной, и удалённой, и действующей.

```
> python decrypto.py
===== GUID @ 0x37e6c8, payload 232 байт =====
# ledger record - fund 1147, opis 1, delo 9
[esp]
bootorder=hd,internal-shell
console=serial,115200
recovery_pin=1147-0091
note=digitization batch 9: originals sealed in the depository
first_record=flag{1147_first_r3c0rd_w4s_f0und}

===== GUID @ 0x37f604, payload 233 байт =====
# ledger record - fund 1147, opis 1, delo 9
[esp]
bootorder=hd,internal-shell
console=serial,115200
recovery_pin=1147-0091
note=digitization batch 9: originals sealed in the depository
first_record=REDACTED-TRANSFERRED-TO-MAIN-VAULT
```

И так тяжким трудом мы получаем наш флааааааг)